Python asyncio Cheat Sheet

Overview

The asyncio library in Python provides tools for writing concurrent code using the async/await syntax. It enables efficient handling of I/O-bound tasks by running them concurrently within a single thread.

Core Concepts

- **Coroutine** – A special function declared with **`async def`** that can pause and resume execution using **`await`**.
- **Event Loop** – The core of **`asyncio`**; it runs and schedules coroutines.
- **Task** – A wrapper around a coroutine that allows it to run concurrently with others.
- **Awaitable** – Any object that can be used with **`await`**, such as coroutines and tasks.

Basic Syntax

```
import asyncio

async def main():
    print('Hello')
    await asyncio.sleep(1)
    print('World')

asyncio.run(main())
```

Running Multiple Coroutines

Use **`asyncio.gather()`** to run coroutines concurrently:

```
import asyncio

async def say_after(delay, text):
    await asyncio.sleep(delay)
    print(text)

async def main():
    await asyncio.gather(
        say_after(1, 'Hello'),
        say_after(2, 'World'))

asyncio.run(main())
```

Creating and Managing Tasks

Use **`asyncio.create_task()`** to start coroutines concurrently:

```
async def worker(name, delay):
    await asyncio.sleep(delay)
    print(f'{name} done')

async def main():
    t1 = asyncio.create_task(worker('A', 1))
    t2 = asyncio.create_task(worker('B', 2))
    await t1
    await t2

asyncio.run(main())
```

Synchronization Primitives

Asyncio provides primitives for controlling access to shared resources:

```
lock = asyncio.Lock()

async with lock:
    # critical section
    ...
```

Other primitives: **`asyncio.Semaphore`**, **`asyncio.Event`**, **`asyncio.Queue`**.

Common asyncio Functions

- **`asyncio.run(coro)`** – Runs a coroutine until it completes.
- **`asyncio.create_task(coro)`** – Schedules a coroutine as a Task.
- **`asyncio.gather(*coros)`** – Runs multiple coroutines concurrently and waits for all to finish.
- **`asyncio.sleep(delay)`** – Non-blocking sleep.
- **`asyncio.wait_for(coro, timeout)`** – Runs a coroutine with a timeout.
- **`asyncio.shield(coro)`** – Protects a coroutine from cancellation.

Error Handling

Handle exceptions within async functions using try/except:

```
async def risky():
    raise ValueError('Something went wrong')

async def main():
```

```
    try:
        await risky()
    except ValueError as e:
        print('Caught:', e)

asyncio.run(main())
```

Best Practices

- Use **`asyncio.run()`** only once at the program's entry point.
- Avoid blocking calls (like **`time.sleep()`**) inside async functions.
- Use **`asyncio.gather()`** instead of manually awaiting each task for efficiency.
- Use locks (**`asyncio.Lock`**) to protect shared resources.
- Combine **`asyncio`** with **`aiohttp`** or **`asyncio.subprocess`** for advanced I/O scenarios.

Debugging Tips

- Enable debug mode: **`asyncio.run(main(), debug=True)`**
- Use **`asyncio.all_tasks()`** to list running tasks.
- Check for un-awaited coroutines — they cause warnings and bugs.
- Use logging to trace task execution.